

DOCUMENTAÇÃO AWS

EMPRESA: TECHSTOCK

TECHSTOCK
AWS FULL

Sumário

1. Introdução	7.4 Integração com Aurora PostgreSQL
2. Objetivos do Projeto	8. Frontend
3. Arquitetura AWS Full	8.1 Hospedagem S3
3.1 Identificação dos Recursos Criados	8.2 Comunicação com Backend
3.2 Endereçamento Utilizado	9. Balanceamento de Carga
4. Topologia da Solução	9.1 Application Load Balancer Backend
5. Configuração da Rede	9.2 Target Groups
5.1 Virtual Private Cloud (VPC)	9.3 Health Checks
5.2 Sub-redes Públicas	10. Monitoramento
5.3 Sub-redes Privadas	10.1 Instância EC2 Monitoring
5.4 Internet Gateway	10.2 Prometheus
5.5 Tabelas de Rotas	10.3 Grafana
6. Banco de Dados	10.4 Dashboard Operacional
6.1 Aurora PostgreSQL	11. Fluxo Completo da Aplicação
6.2 Configuração do Banco	12. Desafios Encontrados
6.3 Estrutura de Dados	13. Resultados Obtidos
7. Backend da Aplicação	14. Lições Aprendidas
7.1 Instância EC2 Backend	15. Evolução do Ambiente
7.2 Aplicação Node.js	16. Conclusão
7.3 APIs Disponíveis	17. Evidências

1. Introdução

O projeto TechStock consiste em uma aplicação de controle de estoque desenvolvida para demonstrar a implementação de uma arquitetura em nuvem baseada em serviços AWS. O ambiente foi projetado seguindo conceitos de alta disponibilidade, segregação de rede, monitoramento e hospedagem de aplicações modernas.

Durante esta etapa do projeto foi realizada a construção completa da infraestrutura AWS, abrangendo rede, computação, banco de dados, balanceamento de carga, monitoramento e hospedagem da interface web.

O objetivo principal foi disponibilizar uma aplicação funcional e monitorada, servindo posteriormente como base para a integração Multi-Cloud AWS + Azure.

2. Objetivos do Projeto

Os objetivos desta implementação foram:

- Construir uma arquitetura AWS funcional e documentada.
 - Hospedar uma aplicação Web utilizando serviços nativos da AWS.
 - Implementar banco de dados gerenciado utilizando Aurora PostgreSQL.
 - Disponibilizar acesso externo através de Load Balancers.
 - Implementar monitoramento utilizando Prometheus e Grafana.
 - Garantir comunicação segura entre os componentes internos.
 - Preparar a infraestrutura para futura integração com Microsoft Azure.
-

3. Arquitetura AWS Full

A arquitetura implementada foi composta pelos seguintes serviços:

- Amazon VPC
- Internet Gateway
- Sub-redes Públicas
- Sub-redes Privadas
- Application Load Balancer (Backend)
- Application Load Balancer (Monitoring)
- Amazon EC2 Backend
- Amazon EC2 Monitoring
- Amazon Aurora PostgreSQL
- Amazon S3
- Prometheus
- Grafana

3.1 Identificação dos Recursos Criados

Rede

Recurso	Finalidade
VPC TechStock	Rede principal do projeto
Public Subnet A	Hospedagem dos ALBs
Public Subnet B	Alta disponibilidade
Private-App	Backend Node.js
Private-DB	Aurora PostgreSQL
Private-Monitoring	Prometheus e Grafana



3.2 Endereçamento Utilizado

Segmento	CIDR
VPC	10.0.0.0/16
Public Subnet A	10.0.0.0/24
Public Subnet B	10.0.1.0/24
Private-App	10.0.10.0/24
Private-DB	10.0.20.0/24

4. Topologia da Solução

Fluxo principal:

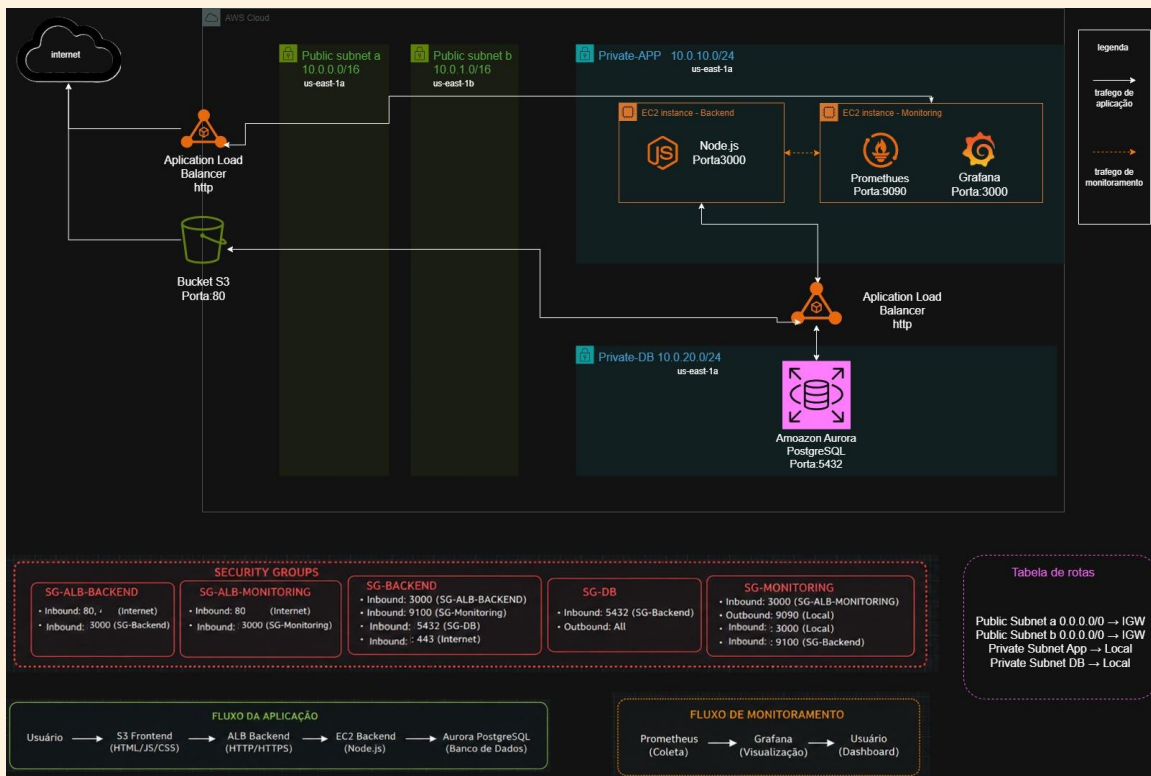
Usuário → Amazon S3 → Application Load Balancer → EC2 Backend → Aurora PostgreSQL

Fluxo de monitoramento:

Prometheus → Endpoint /metrics → Backend

Grafana → Prometheus

Usuário → ALB Monitoring → Grafana



5. Configuração da Rede

5.1 Virtual Private Cloud (VPC)

Foi criada uma VPC dedicada para o projeto TechStock, responsável por isolar logicamente todos os recursos utilizados na solução.

A utilização da VPC permitiu a criação de ambientes segmentados para aplicação, banco de dados e monitoramento.

5.2 Sub-redes Públicas

As sub-redes públicas foram utilizadas para disponibilizar acesso externo aos Application Load Balancers.

Essas sub-redes possuem comunicação com a internet através do Internet Gateway configurado na VPC.

5.3 Sub-redes Privadas

As sub-redes privadas foram utilizadas para hospedar os recursos críticos da solução:

- EC2 Backend
- EC2 Monitoring
- Aurora PostgreSQL

Essa abordagem reduz a exposição direta dos serviços internos.

6. Banco de Dados

6.1 Aurora PostgreSQL

Foi utilizado o Amazon Aurora PostgreSQL como banco de dados principal da aplicação.

O serviço foi implantado em sub-redes privadas e configurado para comunicação exclusiva com a camada de aplicação.

Nome do Cluster

techstock-rds

Instância Writer

techstock-rds-instance-1

Endpoint Writer

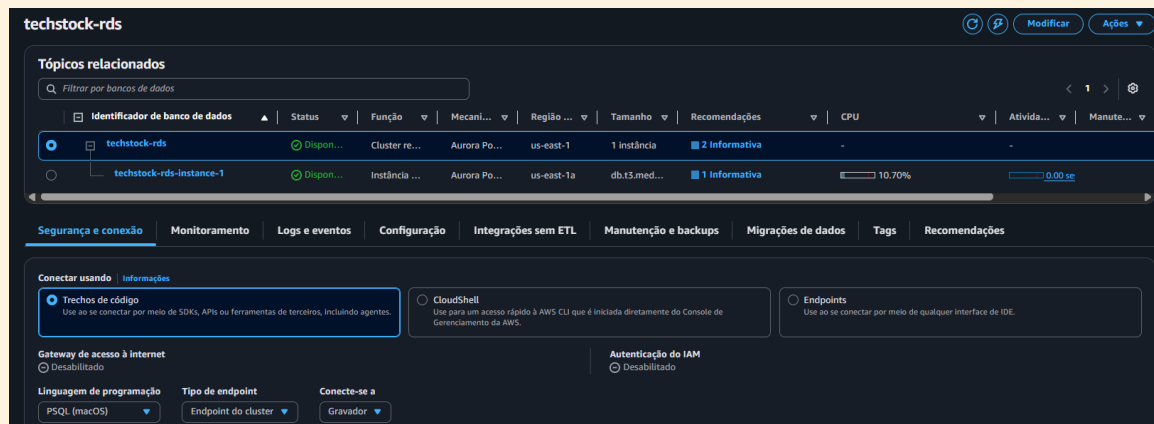
techstock-rds.cluster-ch7crarpzafh.us-east-1.rds.amazonaws.com

Endpoint Reader

techstock-rds.cluster-ro-ch7crarpzafh.us-east-1.rds.amazonaws.com

Porta

5432



6.2 Configuração do Banco

O banco de dados foi configurado para permitir acesso apenas a partir da camada de aplicação através dos Security Groups definidos na arquitetura.

A comunicação foi realizada utilizando PostgreSQL na porta 5432.

6.3 Estrutura de Dados

A base de dados foi inicializada utilizando o arquivo schema.sql disponibilizado no repositório.

Foram criadas as tabelas:

- categorias
- produtos

- movimentos

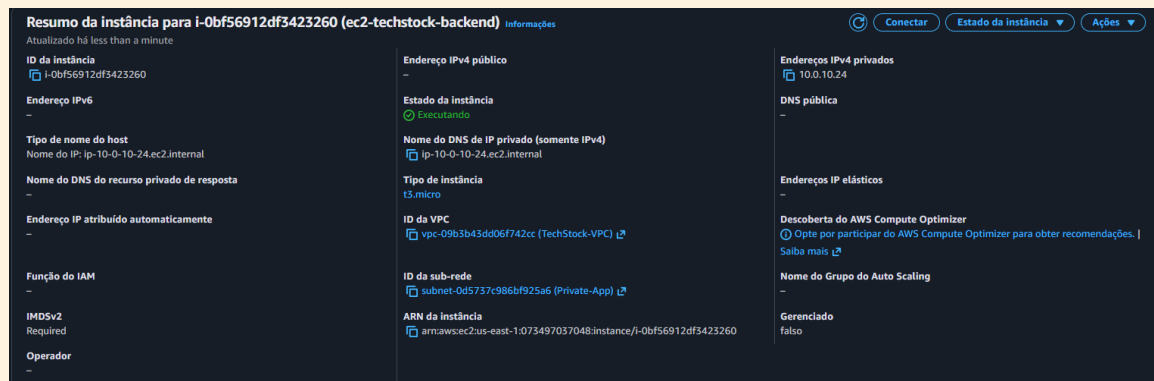
Além da criação de triggers responsáveis pela atualização automática dos registros.

7. Backend da Aplicação

7.1 EC2 Backend

O backend foi implantado em uma instância EC2 Ubuntu.

A aplicação foi desenvolvida em Node.js utilizando Express.



7.2 Aplicação Node.js

Tecnologias Utilizadas

- Node.js
- Express
- PostgreSQL
- Prometheus Client
- Cors
- Helmet
- AWS SDK

A aplicação foi responsável por disponibilizar os endpoints da API, realizar a comunicação com o banco Aurora PostgreSQL e expor métricas para monitoramento através do Prometheus.

7.3 APIs Disponíveis

Durante nossa implementação e testes operacionais, os endpoints efetivamente utilizados para validação da aplicação e monitoramento foram:

- /api/health
- /api/categorias
- /api/produtos
- /api/movimentos
- /api/stats
- /metrics

O endpoint /api/health foi utilizado para verificar a disponibilidade da aplicação, validar a comunicação através do Application Load Balancer e confirmar o funcionamento do backend durante as etapas de implantação e troubleshooting.

O endpoint /metrics foi utilizado pelo Prometheus para coleta das métricas da aplicação, permitindo a visualização dos dados operacionais no Grafana.

Esses endpoints foram fundamentais para os testes de conectividade, validação dos Health Checks e monitoramento contínuo do ambiente.

7.4 Integração com Aurora PostgreSQL

A comunicação entre backend e banco de dados foi realizada através do driver PostgreSQL da aplicação Node.js.

As credenciais e parâmetros de conexão foram definidos por variáveis de ambiente.

```
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)

* Documentation: https://help.ubuntu.com
* Management:   https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Mon Jun 22 07:48:03 UTC 2026

System load: 0.29           Temperature: ~273.1 C
Usage of /: 35.2% of 6.71GB Processes: 116
Memory usage: 26%          Users logged in: 0
Swap usage: 0%             IPv4 address for ens5: 10.0.10.24

* Ubuntu Pro delivers the most comprehensive open source security and
  compliance features.
  https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Mon Jun 22 03:13:57 2026 from 18.206.107.28
ubuntu@ip-10-0-10-24:~$ curl localhost:3000/api/health
{"ok":true,"database":"connected","db":{"ts":"2026-06-22T07:48:10.386Z","ver":"PostgreSQL 17.7 on x86_64-pc-linux-gnu, compiled by x86_64-pc-linux-gnu-gcc (GCC) 10.5.0, 4","uptime_s":256,"uptime":256,"env":"production","secret":"n\u00e3o configurado"}}ubuntu@ip-10-0-10-24:~$
```

8. Frontend

8.1 Amazon S3

O frontend foi hospedado utilizando Amazon S3 em modo Static Website Hosting.

A aplica\u00e7\u00e3o \u00e9 composta pelos arquivos:

- index.html
- style.css
- app.js

TechStock

Controle de Estoque

host: ip-10-0-10-24

API Online

Gratuito

Recorr\u00eancia

Dashboard

Produtos

Movimenta\u00e7\u00f5es

Alertas

TOTAL PRODUTOS

10

ALERTAS

3

VALOR EM ESTOQUE

R\$ 2.984,50

MOV. 24H

0

3 produto(s) abaixo do m\u00ednimo!

Ver alertas

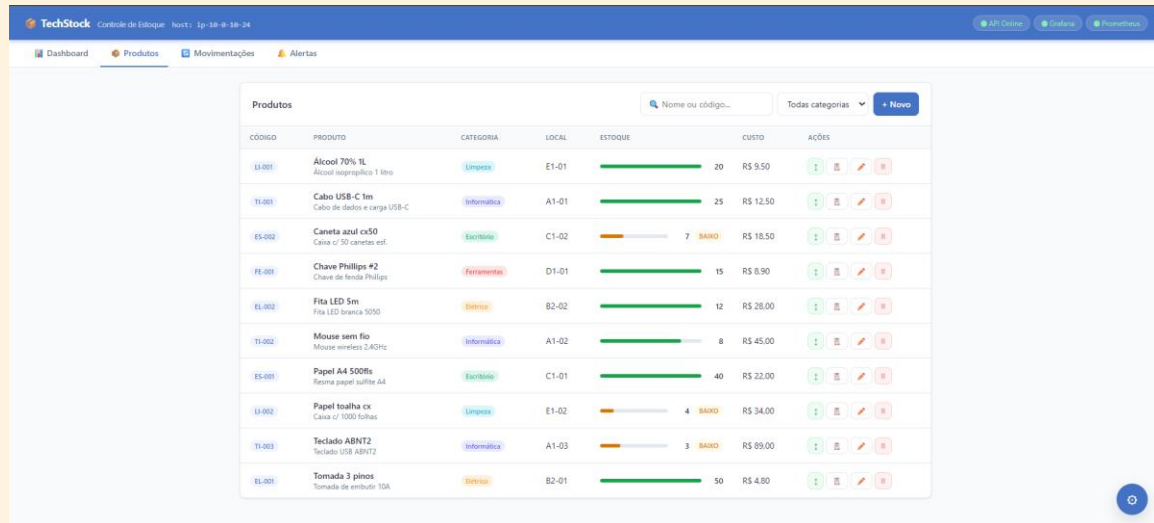
Itens Cr\u00edticos

C\u00d3DIGO	PRODUTO	CATEGORIA	LOCAL	ESTOQUE	
ES-002	Caneta azul cx50	Escrit\u00f3rio	C1-02		7 BAIXO
LI-002	Papel toalha cx	Limpeza	E1-02		4 BAIXO
TI-003	Teclado ABNT2	Inform\u00e1tica	A1-03		3 BAIXO

8.2 Integração com Backend

O frontend permite configurar dinamicamente o DNS do Application Load Balancer responsável pelo backend.

Dessa forma a aplicação realiza chamadas diretamente para as APIs disponibilizadas.



The screenshot displays the 'Produtos' (Products) section of the TechStock application. The interface includes a top navigation bar with 'Dashboard', 'Produtos', 'Movimentações', and 'Alertas'. A search bar and a '+ Novo' button are located at the top right of the table. The table itself has columns for 'CÓDIGO', 'PRODUTO', 'CATEGORIA', 'LOCAL', 'ESTOQUE', 'CUSTO', and 'AÇÕES'. Each row represents a product with its details and a set of action icons (add, edit, delete).

CÓDIGO	PRODUTO	CATEGORIA	LOCAL	ESTOQUE	CUSTO	AÇÕES
LI-001	Alcool 70% 1L Alcool isopropílico 1 litro	Limpieza	E1-01	20	R\$ 9.50	+ ✎ ✖
TI-001	Cabo USB-C 1m Cabo de dados e carga USB-C	Informática	A1-01	25	R\$ 12.50	+ ✎ ✖
ES-002	Caneta azul cx50 Caneta c/ 50 canetas esf.	Escritório	C1-02	7 SAÍDO	R\$ 18.50	+ ✎ ✖
FE-001	Chave Phillips #2 Chave de fenda Phillips	Ferramentas	D1-01	15	R\$ 8.90	+ ✎ ✖
EL-002	Fita LED 5m Fita LED branca 5050	Iluminação	B2-02	12	R\$ 28.00	+ ✎ ✖
TI-002	Mouse sem fio Mouse wireless 2.4GHz	Informática	A1-02	8	R\$ 45.00	+ ✎ ✖
ES-001	Papel A4 500fs Papel papel sulfite A4	Escritório	C1-01	40	R\$ 22.00	+ ✎ ✖
LI-002	Papel toalha cx Canva c/ 1000 folhas	Limpieza	E1-02	4 SAÍDO	R\$ 34.00	+ ✎ ✖
TI-003	Teclado ABNT2 Teclado USB ABNT2	Informática	A1-03	3 SAÍDO	R\$ 89.00	+ ✎ ✖
EL-001	Tomada 3 pinos Tomada de embudo 10A	Iluminação	B2-01	50	R\$ 4.80	+ ✎ ✖

9. Balanceamento de Carga

Foram implementados dois Application Load Balancers independentes:

- Backend
- Monitoring

A separação permitiu isolar o tráfego operacional do tráfego de monitoramento.

Load balancers (2) O que há de novo?

Ações

Criar load balancer

O Elastic Load Balancing mede automaticamente a capacidade do seu balanceador de carga em resposta a alterações no tráfego de entrada.

Filtrar balanceadores de carga

<

1

>

☐	Nome	Estado	Tipo	Esquema	Tipo de endereço IP	ID da VPC	Zonas de disponibi
☐	alb-techstock	Ativo	application	Internet-facing	IPv4	vpc-09b3b43dd06f742cc	2 Zonas de disponibilidade
☐	alb-monitoring	Ativo	application	Internet-facing	IPv4	vpc-09b3b43dd06f742cc	2 Zonas de disponibilidade

10. Monitoramento

10.1 Instância EC2 Monitoring

A instância de monitoramento foi dedicada à execução das ferramentas de observabilidade da solução.

Funções

- Prometheus
- Grafana

10.2 Prometheus

O Prometheus foi configurado para coletar métricas diretamente do endpoint /metrics da aplicação.

Endpoint Monitorado

http://54.82.43.238:9090/

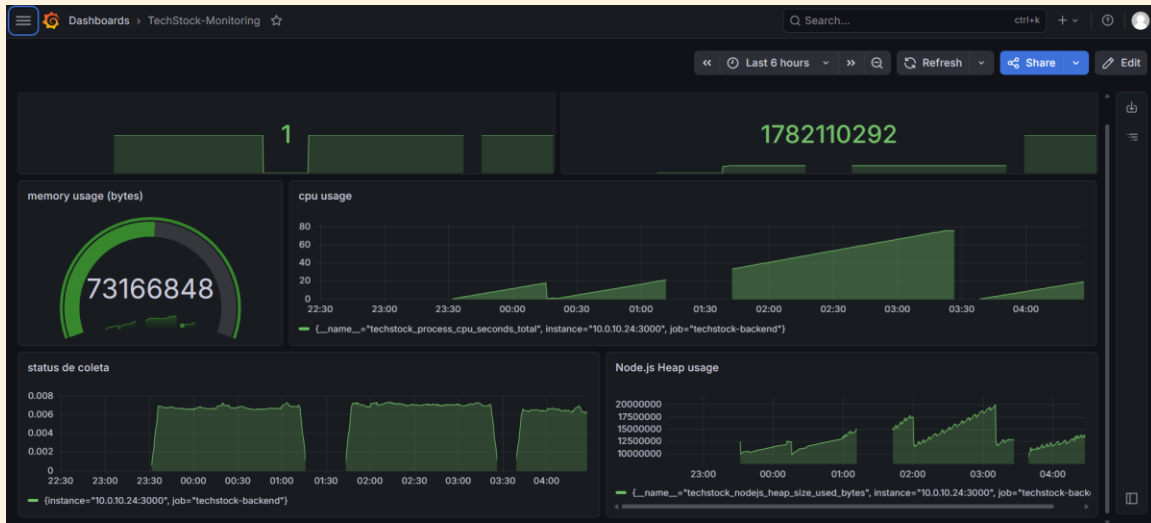
Entre as métricas coletadas estão:

- CPU
- Memória
- Heap do Node.js
- Disponibilidade da aplicação

10.3 Grafana

O Grafana foi integrado ao Prometheus para visualização das métricas.

Foi criado um dashboard operacional contendo indicadores de disponibilidade e utilização da aplicação.



10.4 Dashboard Operacional

Indicadores Implementados

Backend Online

up

Uso de CPU

rate(techstock_process_cpu_seconds_total[5m]))

Uso de Memória

techstock_process_resident_memory_bytes

Heap Node.js

techstock_nodejs_heap_size_used_bytes

Inicialização da Aplicação

techstock_process_start_time_seconds

11. Fluxo Completo da Aplicação

1. Usuário acessa o frontend hospedado no S3.
 2. Frontend envia requisições para o ALB Backend.
 3. O ALB encaminha as requisições para a EC2 Backend.
 4. A aplicação consulta o Aurora PostgreSQL.
 5. O retorno é exibido ao usuário.
 6. O Prometheus coleta métricas da aplicação.
 7. O Grafana apresenta as métricas em tempo real.
-

12. Desafios Encontrados

Durante a implementação foram encontrados desafios relacionados a:

- Configuração de Security Groups.
- Health Checks dos Target Groups.
- Comunicação entre instâncias privadas.
- Dependências Node.js ausentes.
- Configuração de monitoramento.

Todos os problemas foram solucionados através de validações de conectividade, revisão das regras de segurança e testes incrementais dos componentes.

13. Resultados Obtidos

Ao final da implementação foram alcançados os seguintes resultados:

- Ambiente AWS totalmente funcional.
 - Aplicação operacional.
 - Banco de dados integrado.
 - Frontend acessível publicamente.
 - Monitoramento em tempo real.
 - Dashboard operacional.
 - Arquitetura pronta para integração Multi-Cloud.
-

14. Lições Aprendidas

Durante a implementação da arquitetura AWS Full foram adquiridos conhecimentos práticos que vão além da simples criação dos recursos.

Security Groups

Foi observado que uma configuração incorreta das regras de entrada pode impedir completamente a comunicação entre componentes, mesmo quando todos os demais recursos estão configurados corretamente.

Aprendizados

- Validar origem e destino das regras.
 - Utilizar grupos específicos para cada serviço.
 - Evitar regras excessivamente permissivas em produção.
-

Target Groups

Foi identificado que um Target Group pode apresentar status Unhealthy mesmo quando a aplicação parece estar operacional.

Aprendizados

- Verificar o endpoint utilizado no Health Check.
 - Confirmar portas de escuta da aplicação.
 - Testar diretamente com curl antes de investigar o Load Balancer.
-

Application Load Balancer

Durante os testes foram observados erros HTTP 503 e HTTP 504.

Aprendizados

- Erro 503 normalmente indica ausência de destinos saudáveis.
 - Erro 504 normalmente indica falha de comunicação entre ALB e instância.
-

Monitoramento

A implementação do Prometheus demonstrou a importância da observabilidade.

Aprendizados

- Não basta que a aplicação funcione.
 - É necessário acompanhar CPU, memória e disponibilidade.
-

Documentação

Foi constatado que registrar nomes dos recursos, IPs, DNS e configurações reduz significativamente o tempo necessário para localizar e corrigir problemas.

Além de servir como registro técnico do projeto, a documentação torna-se uma referência para futuras manutenções e para equipes que precisem compreender rapidamente a arquitetura implementada.

15. Evolução do Ambiente

Fase 1

Criação da VPC

Fase 2

Configuração das Subnets

Fase 3

Implantação do Aurora PostgreSQL

Fase 4

Implantação do Backend Node.js

Fase 5

Configuração do Application Load Balancer

Fase 6

Publicação do Frontend em S3

Fase 7

Implantação do Prometheus

Fase 8

Implantação do Grafana

Fase 9

Criação do Dashboard Operacional

Fase 10

Preparação para integração AWS + Azure

Esta linha do tempo representa a evolução gradual da infraestrutura e demonstra o processo de aprendizado e amadurecimento técnico durante o desenvolvimento do projeto.

16. Conclusão

A implementação da arquitetura AWS Full do projeto TechStock demonstrou a utilização integrada dos principais serviços da AWS para construção de uma aplicação moderna baseada em nuvem.

A infraestrutura criada fornece escalabilidade, monitoramento, isolamento de rede e capacidade de expansão para cenários Multi-Cloud, servindo como base para as próximas etapas do projeto envolvendo integração com Microsoft Azure.

Além dos resultados técnicos obtidos, o projeto proporcionou experiência prática em arquitetura de redes, banco de dados gerenciado, monitoramento, troubleshooting e documentação de ambientes em nuvem, consolidando conhecimentos fundamentais para futuras implementações profissionais.

17. Evidências

Inserir nesta seção todos os prints capturados durante a implementação:

- Arquitetura AWS
- VPC
- Sub-redes
- CIDRs da rede
- Aurora PostgreSQL
- Cluster Aurora
- EC2 Backend
- EC2 Monitoring
- S3

- Application Load Balancers
- Target Groups
- Security Groups
- Prometheus Targets
- Grafana
- Dashboard Operacional
- Fluxo Final da Aplicação
- Ambiente Completo AWS
- Dashboard Final

Dashboard.

TECHSTOCK
AWS FULL